

ARSENAL TECH

L'ÉLITE 2026

PARTIE II · DATA ENGINEERING & INTELLIGENCE MASSIVE

CHAPITRE 3

Pipelines de Données

"God-Mode"

Ingestion Massive · Apache Spark · Cassandra · MongoDB · Neo4j

402 PB mondial Data générée/jour (2026)	1 M+ par nœud Spark : records/sec (cluster)	<1 ms P99 production Cassandra : latence write	~60% avec pipeline optimisé Réduction coût vs SQL pur
--	--	--	--

Introduction : Le Pipeline comme Système Nerveux de la Donnée

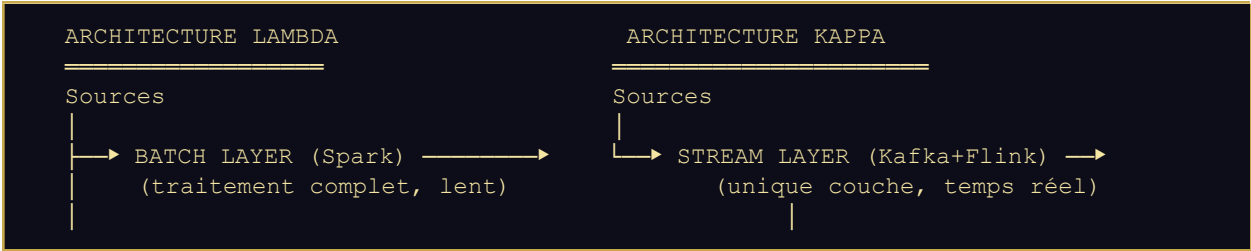
En 2026, la valeur d'une entreprise technologique se mesure moins à la qualité de son code qu'à la qualité de ses données. Un pipeline de données mal conçu est l'équivalent d'un système circulatoire obstrué : l'intelligence s'accumule quelque part, ne circule nulle part, et l'organisation prend des décisions à l'aveugle. Le Data Engineer God-Mode est celui qui conçoit des systèmes où la donnée brute, quelle que soit sa source, son volume ou sa vélocité, est transformée en intelligence exploitable en temps quasi-réel.

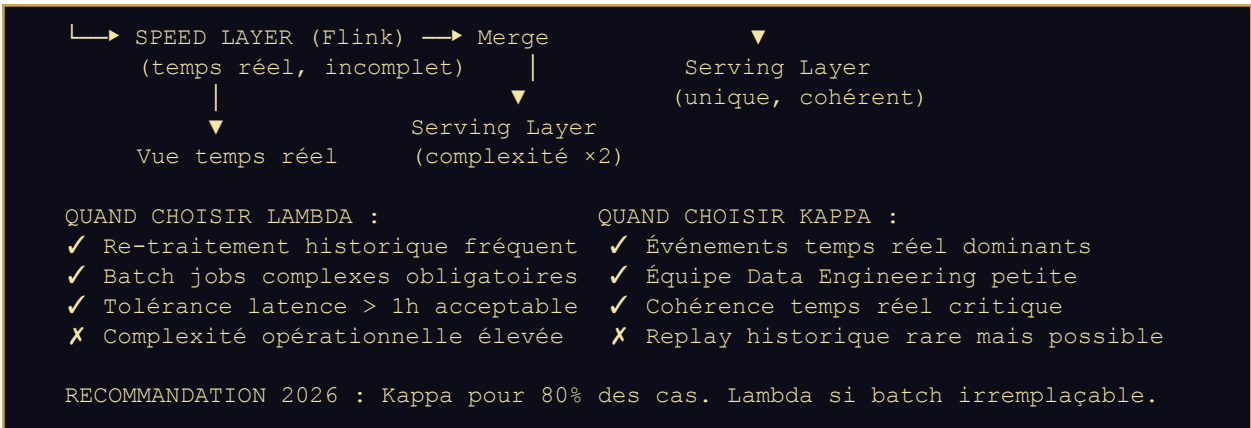
Ce chapitre couvre la triade complète : l'ingestion et le stockage structuré (la bouche du pipeline), Apache Spark pour le traitement massif en mémoire (le cerveau), et les bases NoSQL spécialisées — Cassandra, MongoDB, Neo4j — pour le stockage et la requête à l'échelle (la mémoire long terme). Chaque section est accompagnée d'implémentations production-grade que vous pouvez déployer demain.

► Sous-Partie 1 : Ingestion Massive — De la Donnée Brute au Stockage Structuré

◆ L'Architecture Lambda vs Kappa : Le Choix Fondamental

Avant d'écrire une seule ligne de code d'ingestion, l'architecte Data doit trancher entre deux paradigmes fondamentaux qui conditionnent l'ensemble de l'architecture. Ce choix engage l'organisation pour 3 à 5 ans.



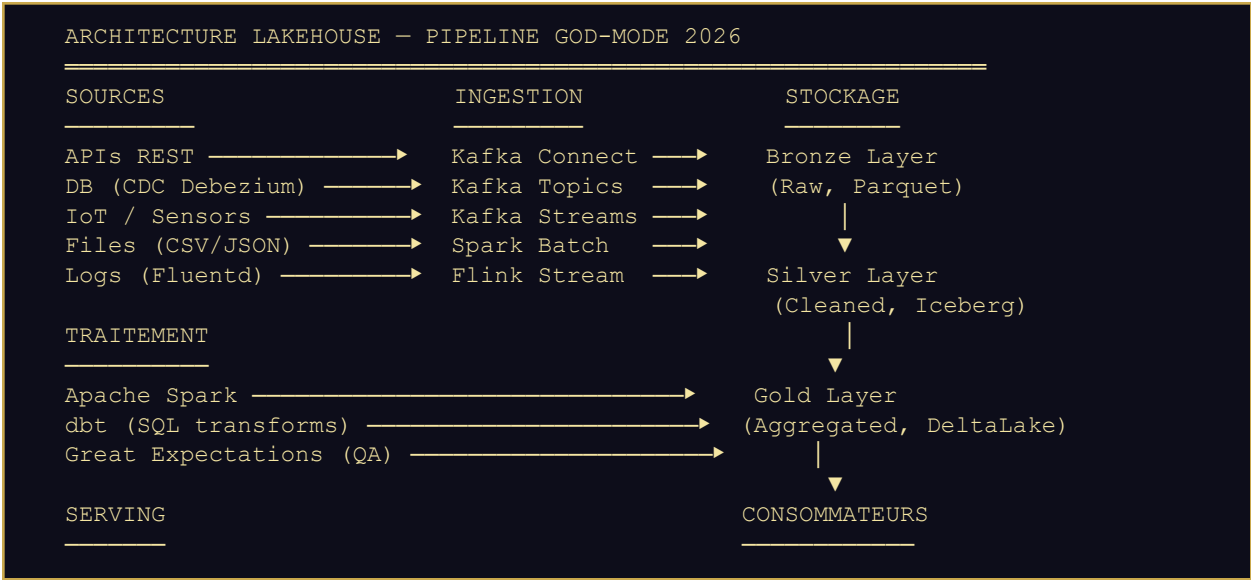


◆ Les 4 Patterns d'Ingestion et Leurs Cas d'Usage

Pattern	Outil Référence	Volume	Latence	Cas d'Usage Typique
Batch	Apache Spark / dbt	Très élevé	> 1h	Data warehouse, rapports quotidiens
Micro-batch	Spark Structured Streaming	Élevé	10-60s	Tableaux de bord near-real-time
Stream pur	Apache Flink / Kafka Streams	Variable	< 1s	Fraude temps réel, IoT, alertes
Change Data Capture	Debezium + Kafka	Modéré	< 5s	Sync DB → Data Lake sans disruption

◆ Architecture Lakehouse Moderne : Le Standard 2026

Le Data Lakehouse fusionne la flexibilité du Data Lake (stockage brut pas cher) avec la fiabilité du Data Warehouse (transactions ACID, schémas). En 2026, Delta Lake (Databricks), Apache Iceberg et Apache Hudi sont les trois formats de table qui rendent cela possible. Iceberg est devenu le standard de facto pour les architectures open-source.



```

Trino / Athena —————> BI (Metabase)
Cassandra (time-series) —————> APIs ML
Elasticsearch —————> Data Scientists
=====
ORCHESTRATION : Apache Airflow 2.x / Prefect
MONITORING    : Great Expectations + Monte Carlo (Data Observability)
CATALOG       : Apache Atlas / DataHub

```

◆ Implémentation : Pipeline CDC avec Debezium → Kafka → Iceberg

Le Change Data Capture (CDC) est le pattern d'ingestion le plus puissant pour synchroniser une base de données de production vers votre Data Lake sans impact sur les performances. Voici l'implémentation complète avec Debezium, Kafka et Apache Iceberg :

* YAML + JSON — DEBEZIUM CDC PIPELINE CONFIG

```

# — docker-compose : Stack CDC complète —————
version: '3.8'
services:

  # — Source : PostgreSQL avec WAL activé —————
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: production_db
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    command:
      - postgres
      - -c wal_level=logical          # CRITIQUE : Active le Write-Ahead Logging
      - -c max_replication_slots=4
      - -c max_wal_senders=4

  # — CDC Engine : Debezium Connector —————
  debezium:
    image: debezium/connect:2.5
    depends_on: [kafka, postgres]
    environment:
      BOOTSTRAP_SERVERS: kafka:9092
      GROUP_ID: debezium-cdc-group
      CONFIG_STORAGE_TOPIC: debezium.configs
      OFFSET_STORAGE_TOPIC: debezium.offsets
      STATUS_STORAGE_TOPIC: debezium.status

  ---
  # — Debezium Connector Config : Capture toutes les tables —————
  POST /connectors HTTP/1.1
  Content-Type: application/json
  {
    "name": "postgres-cdc-connector",
    "config": {
      "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
      "database.hostname": "postgres",
      "database.port": "5432",
      "database.user": "debezium_user",
      "database.password": "${DEBEZIUM_PASSWORD}",
      "database.dbname": "production_db",
      "database.server.name": "prod",
      "table.include.list": "public.transactions,public.users,public.products",

```

```

    "plugin.name": "pgoutput",

    "transforms": "unwrap,addTimestamp",
    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "op,source.ts_ms",

    "key.converter": "io.confluent.kafka.serializers.KafkaAvroSerializer",
    "value.converter": "io.confluent.kafka.serializers.KafkaAvroSerializer",
    "schema.registry.url": "http://schema-registry:8081"
  }
}

# Résultat : Topic Kafka 'prod.public.transactions' reçoit chaque INSERT/UPDATE/DELETE
# Format : { op: 'c'|'u'|'d', before: {...}, after: {...}, source.ts_ms: 1712345678 }

```

♦ PYSPARK — STRUCTURED STREAMING CDC → APACHE ICEBERG

```

# — Spark Job : CDC Kafka → Iceberg (Bronze Layer) —
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, current_timestamp
from pyspark.sql.types import StructType, StringType, LongType

spark = SparkSession.builder \
    .appName('cdc-kafka-to-iceberg') \
    .config('spark.sql.extensions',
'org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions') \
    .config('spark.sql.catalog.local', 'org.apache.iceberg.spark.SparkCatalog') \
    .config('spark.sql.catalog.local.type', 'hadoop') \
    .config('spark.sql.catalog.local.warehouse', 's3a://data-lake/warehouse') \
    .getOrCreate()

# — Schéma de l'événement CDC Debezium —
cdc_schema = StructType() \
    .add('op', StringType()) \
    .add('source_ts', LongType()) \
    .add('after', StructType()
        .add('id', StringType())
        .add('user_id', StringType())
        .add('amount', StringType())
        .add('currency', StringType())
        .add('status', StringType())
        .add('created_at', LongType())
    )

# — Lecture Stream Kafka —
raw_stream = spark.readStream \
    .format('kafka') \
    .option('kafka.bootstrap.servers', 'kafka:9092') \
    .option('subscribe', 'prod.public.transactions') \
    .option('startingOffsets', 'earliest') \
    .option('maxOffsetsPerTrigger', 100_000) # Micro-batch: 100K events/trigger \
    .load()

# — Parsing et enrichissement —
parsed = raw_stream \
    .select(from_json(col('value').cast('string'), cdc_schema).alias('cdc')) \
    .select(
        col('cdc.op').alias('operation'),

```

```

        col('cdc.source_ts').alias('event_ts'),
        col('cdc.after.id').alias('transaction_id'),
        col('cdc.after.user_id'),
        col('cdc.after.amount').cast('decimal(18,2)'),
        col('cdc.after.currency'),
        col('cdc.after.status'),
        current_timestamp().alias('ingested_at') # Watermark d'ingestion
    ) \
    .filter(col('cdc.after').isNotNull()) # Filtre les DELETE pour Bronze

# — Écriture vers Iceberg (Bronze Layer, append-only) —————
query = parsed.writeStream \
    .format('iceberg') \
    .outputMode('append') \
    .option('path', 'local.bronze.transactions_raw') \
    .option('checkpointLocation', 's3a://checkpoints/bronze/transactions') \
    .trigger(processingTime='30 seconds') \
    .start()

query.awaitTermination()

```

◆ Qualité des Données : Great Expectations en Pipeline

Un pipeline sans validation de qualité n'est pas un pipeline — c'est un conduit à pollution. Great Expectations est la bibliothèque de référence pour définir et exécuter des contrats de données en production :

✦ PYTHON — GREAT EXPECTATIONS DATA CONTRACT

```

# — Great Expectations : Contrat de Données Silver Layer —————
import great_expectations as ge
from great_expectations.core.batch import RuntimeBatchRequest

context = ge.get_context()

# — Définition du contrat de données (Expectation Suite) —————
suite = context.create_expectation_suite('transactions.silver.contract')

validator = context.get_validator(
    batch_request=RuntimeBatchRequest(
        datasource_name='spark_datasource',
        data_connector_name='runtime_connector',
        data_asset_name='transactions_silver',
        runtime_parameters={'batch_data': df},
    ),
    expectation_suite=suite
)

# — Règles de qualité : Ce que DOIT respecter la donnée —————
validator.expect_column_to_exist('transaction_id')
validator.expect_column_values_to_be_unique('transaction_id')
validator.expect_column_values_to_not_be_null('transaction_id')
validator.expect_column_values_to_not_be_null('user_id')

# Montant : Entre 0 et 10 000 000 (XOF) — refuse les anomalies
validator.expect_column_values_to_be_between('amount', min_value=0, max_value=10_000_000)

# Devise : Uniquement les devises UEMOA/CEDEAO
validator.expect_column_values_to_be_in_set('currency',
['XOF', 'XAF', 'GHS', 'NGN', 'KES', 'USD', 'EUR'])

```

```
# Status : Enum strict
validator.expect_column_values_to_be_in_set('status',
['PENDING', 'SUCCESS', 'FAILED', 'REFUNDED'])

# Fraîcheur : Pas de données de plus de 24h dans le pipeline temps réel
validator.expect_column_values_to_be_between(
    'event_ts',
    min_value=int((datetime.utcnow() - timedelta(hours=24)).timestamp() * 1000),
    max_value=int(datetime.utcnow().timestamp() * 1000)
)

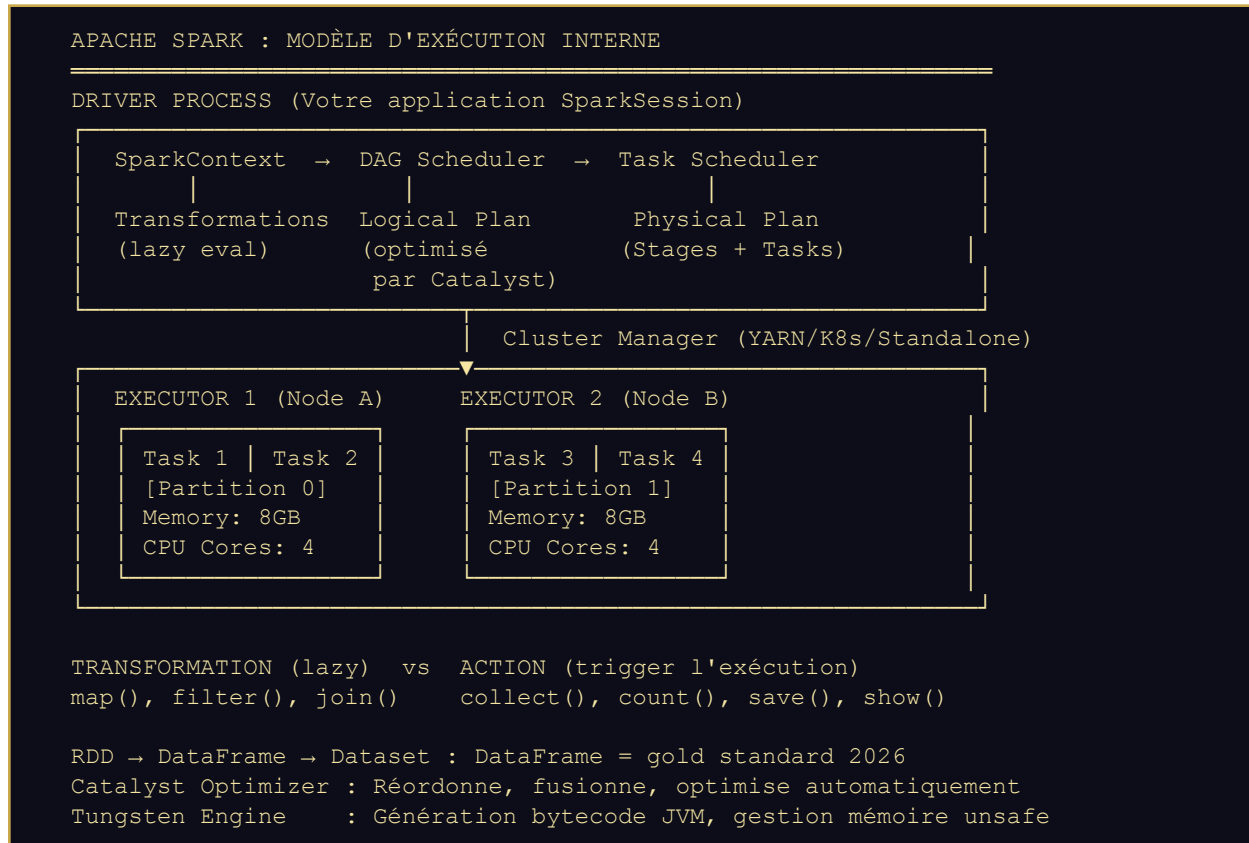
# — Exécution et décision : Bloquer si qualité insuffisante —
results = validator.validate()

if not results.success:
    failed = [r for r in results.results if not r.success]
    # Alerter + envoyer la donnée vers une 'quarantine queue'
    kafka_producer.send('data.quality.failures', {
        'pipeline': 'transactions.silver',
        'failed_checks': len(failed),
        'details': [str(r.expectation_config) for r in failed],
        'timestamp': datetime.utcnow().isoformat()
    })
    raise DataQualityException(f'{len(failed)} quality checks failed — batch quarantined')
```

► Sous-Partie 2 : Maîtrise d'Apache Spark — Analyse In-Memory pour le Big Data

◆ Anatomie de Spark : Ce que Tout Ingénieur Senior Doit Visualiser

Apache Spark 3.5 en 2026 n'est plus simplement un moteur de traitement — c'est une plateforme unifiée pour le batch, le streaming, le ML (MLlib) et le SQL distribué. Mais sa maîtrise exige de comprendre son modèle d'exécution interne, sans quoi vous écrirez du code qui semble fonctionner en dev et s'effondre en production à 10TB.



◆ Les 5 Erreurs de Performance Spark les Plus Coûteuses

● ANTI-PATTERNS SPARK — PERTES DE PERFORMANCE CRITIQUES

1. DATA SKEW : 1 partition à 100GB, 99 à 100MB. Solution : Salting key + repartition(salted_key).
2. SHUFFLE EXCESSIF : join() sans broadcast sur petite table → réseau saturé. Solution : broadcast(small_df).
3. COLLECT() sur grand dataset : Rapatrie tout en mémoire Driver → OOM crash. Toujours sampler ou écrire.
4. UDF Python (non-vectorisées) : Sérialisation Python ↔ JVM ×10 plus lent que SQL natif. Solution : pandas_udf.
5. REPARTITION vs COALESCE : repartition() = full shuffle. Coalesce() = pas de shuffle, pour réduire.

◆ Spark Avancé : Optimisations Production-Grade

♦ PYSPARK — GOLD LAYER AVEC AQE + BROADCAST JOIN + PANDAS UDF

```
# — SPARK CONFIG : Cluster production 20 nœuds × 32 cores × 128GB —
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName('gold-layer-aggregations') \

    # — Mémoire : 80% RAM pour Spark, 20% overhead —
    .config('spark.executor.memory', '96g') \
    .config('spark.executor.cores', '8') \
    .config('spark.executor.instances', '20') \
    .config('spark.driver.memory', '16g') \

    # — Adaptive Query Execution (AQE) : Auto-optimisation —
    .config('spark.sql.adaptive.enabled', 'true') \
    .config('spark.sql.adaptive.coalescePartitions.enabled', 'true') \
    .config('spark.sql.adaptive.skewJoin.enabled', 'true') \
    .config('spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes', '256MB') \

    # — Shuffle optimisé —
    .config('spark.sql.shuffle.partitions', '400') \
    .config('spark.serializer', 'org.apache.spark.serializer.KryoSerializer') \

    # — Iceberg catalog —
    .config('spark.sql.extensions',
'org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions') \
    .getOrCreate()

# — Lecture Gold Layer depuis Iceberg —
transactions = spark.table('local.silver.transactions')
users         = spark.table('local.silver.users')

# — BROADCAST JOIN : Petite table (<= 10MB) broadcastée sur tous les exécuteurs
# Évite le shuffle de la grande table – gain de vitesse ×5 à ×20
users_small = spark.table('local.silver.user_tiers') \
    .select('user_id', 'tier', 'country') \
    .filter(F.col('active') == True)

enriched = transactions.join(
    F.broadcast(users_small), # ← Force le broadcast
    on='user_id',
    how='left'
)

# — WINDOW FUNCTIONS : KPIs par utilisateur (30 jours glissants) —
window_30d = Window \
    .partitionBy('user_id') \
    .orderBy(F.col('event_ts').cast('long')) \
    .rangeBetween(-30 * 86400 * 1000, 0) # 30 jours en ms

gold_df = enriched \
    .filter(F.col('status') == 'SUCCESS') \
    .withColumn('rolling_30d_total', F.sum('amount').over(window_30d)) \
    .withColumn('rolling_30d_count', F.count('transaction_id').over(window_30d)) \
    .withColumn('rolling_30d_avg', F.avg('amount').over(window_30d)) \
```

```

        .withColumn('rank_in_tier',
            F.rank().over(Window.partitionBy('tier').orderBy(F.desc('rolling_30d_total'))))

# — PANDAS UDF : Calcul vectorisé (x10 vs Python UDF standard) —
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf('double') # Vectorisé via Apache Arrow — pas de sérialisation ligne par ligne
def fraud_risk_score(amount: pd.Series, country: pd.Series) -> pd.Series:
    """Score de risque fraude — exécuté sur l'executor, vectorisé"""
    base_score = (amount / 500_000).clip(0, 1) * 50 # 50% basé sur montant
    country_risk = country.map({'NG': 0.3, 'GH': 0.1, 'SN': 0.05}).fillna(0.2) * 50
    return (base_score + country_risk).clip(0, 100)

final_gold = gold_df.withColumn(
    'fraud_risk_score',
    fraud_risk_score(F.col('amount'), F.col('country'))
)

# — ÉCRITURE : Partitionnement intelligent pour performance lecture —
final_gold.write \
    .format('iceberg') \
    .mode('overwrite') \
    .partitionBy('country', F.date_trunc('month', F.col('event_ts'))) \
    .option('write.target-file-size-bytes', 134_217_728) # 128MB par fichier \
    .save('local.gold.user_transaction_kpis')

```

◆ Étude de Cas : OPay Nigeria — Pipeline Spark à 500M Transactions/Jour

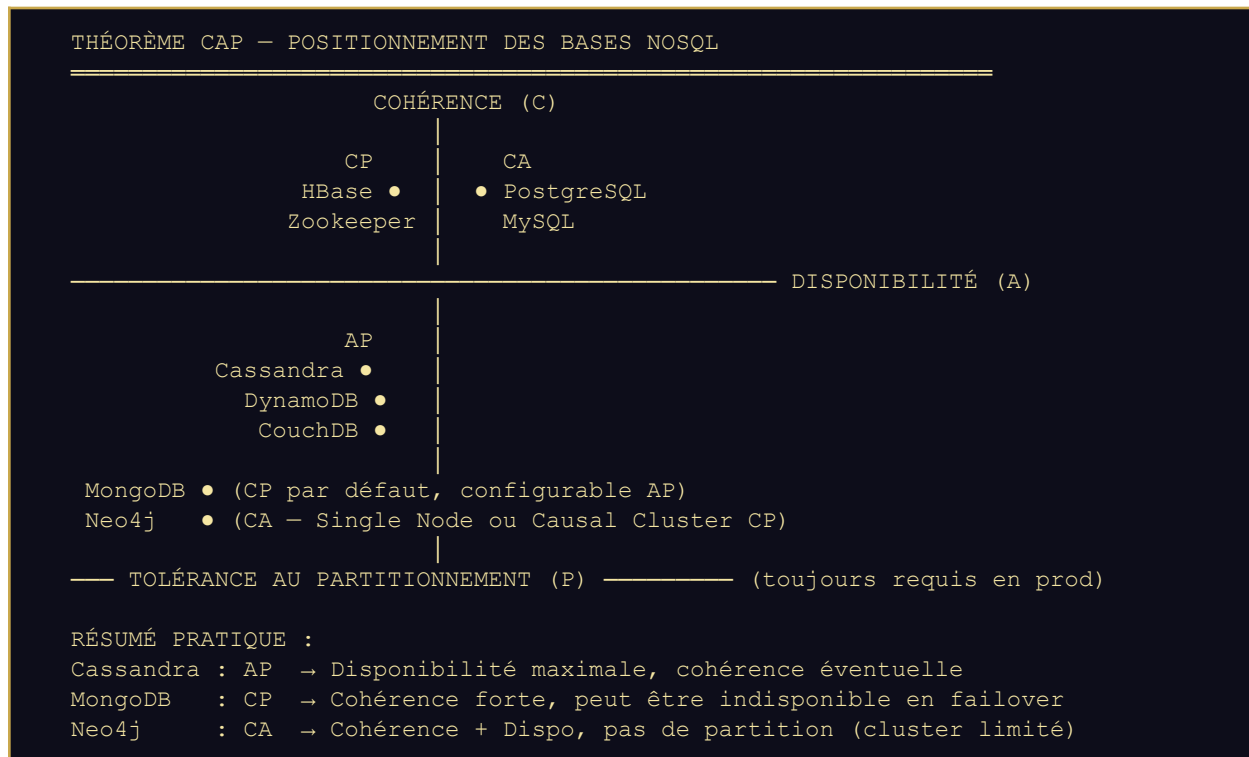
● CASE STUDY — OPAY NIGERIA

Contexte : Fintech mobile money, 40M+ utilisateurs actifs, Nigeria + Égypte + Éthiopie.
 Volume : ~500M transactions/jour → 5,8K transactions/sec en moyenne, pics à 50K/sec.
 Pipeline : Kafka (ingestion) → Spark Structured Streaming (micro-batch 10s) → Iceberg.
 Infra Spark : Cluster AWS EMR, 200 nœuds r6gd.4xlarge (16 cores, 128GB RAM chacun).
 Optimisation clé : Partition pruning par date + country → scan de 2% des données seulement.
 Résultat : Dashboard de risque fraude mis à jour toutes les 30s (vs 4h avec batch pur).
 Stack complémentaire : Trino pour requêtes ad-hoc BI, Metabase pour les dashboards.
 Leçon architecture : Le bon partitionnement Iceberg vaut 10x plus que l'hardware.

► Sous-Partie 3 : NoSQL de Puissance — Cassandra, MongoDB, Neo4j

◆ Le Théorème CAP : La Grille de Décision Fondamentale

Le théorème CAP (Brewer, 2000) reste la boussole de tout architecte qui choisit une base de données distribuée. Il stipule qu'un système distribué ne peut garantir simultanément que 2 des 3 propriétés suivantes : Cohérence (C), Disponibilité (A), Tolérance au Partitionnement (P). En 2026, comprendre où se positionne chaque NoSQL sur ce spectre est non négociable.



◆ Apache Cassandra : L'Indestructible pour le Time-Series et l'IoT

Cassandra est le choix absolu lorsque vous avez besoin d'écriture massive, de disponibilité 99.999% et de scalabilité linéaire. Sa particularité : il n'y a pas de master — tous les nœuds sont égaux (architecture peer-to-peer). Supprimer un nœud ne dégrade pas le service. C'est pour cela que Netflix l'utilise pour stocker toutes les métadonnées de streaming.

```
* CQL — CASSANDRA DATA MODELING PRODUCTION-GRADE

-- — CQL : Modélisation Cassandra pour données de transactions —
-- RÈGLE D'OR : Modéliser selon les queries, PAS selon les entités

-- — Query 1 : "Donner-moi les 100 dernières transactions d'un user"
CREATE TABLE IF NOT EXISTS fintech.transactions_by_user (
  user_id          UUID,
  transaction_ts    TIMESTAMP,    -- Partie de la clé de partition
  transaction_id    UUID,
  amount            DECIMAL,
  currency          TEXT,
  merchant_id       UUID,
```

```

    status          TEXT,
    -- Clustering key : ORDERED BY ts DESC → range query rapide
    PRIMARY KEY ((user_id), transaction_ts, transaction_id)
) WITH CLUSTERING ORDER BY (transaction_ts DESC, transaction_id ASC)
AND compaction = {
    'class': 'TimeWindowCompactionStrategy',
    'compaction_window_unit': 'DAYS',
    'compaction_window_size': 7    -- Fenêtre de 7 jours, optimal time-series
}
AND gc_grace_seconds = 86400    -- Récupération tombstones après 1 jour
AND default_time_to_live = 7776000; -- TTL 90 jours automatique

-- — Query 2 : "Toutes les transactions d'un user ce mois-ci"
-- Partition par (user_id, mois) évite les large partitions (>100MB)
CREATE TABLE IF NOT EXISTS fintech.transactions_by_user_month (
    user_id          UUID,
    month_bucket     TEXT,          -- '2026-03' - évite partition gigantesque
    transaction_ts   TIMESTAMP,
    transaction_id   UUID,
    amount           DECIMAL,
    status           TEXT,
    PRIMARY KEY ((user_id, month_bucket), transaction_ts, transaction_id)
) WITH CLUSTERING ORDER BY (transaction_ts DESC);

-- — INSERT avec TTL et timestamp explicite —————
INSERT INTO fintech.transactions_by_user (
    user_id, transaction_ts, transaction_id, amount, currency, status
) VALUES (
    550e8400-e29b-41d4-a716-446655440000,
    '2026-03-12 14:23:00+0000',
    uuid(),
    15000.00, 'XOF', 'SUCCESS'
) USING TIMESTAMP 1741787000000000 -- Microseconds - idempotent writes
AND TTL 7776000;                  -- 90 jours

-- — SELECT : Range query sur time-series —————
SELECT transaction_id, amount, status, transaction_ts
FROM fintech.transactions_by_user
WHERE user_id = 550e8400-e29b-41d4-a716-446655440000
    AND transaction_ts >= '2026-03-01 00:00:00'
    AND transaction_ts < '2026-04-01 00:00:00'
ORDER BY transaction_ts DESC
LIMIT 100;
-- Performance : < 1ms P99 - lecture séquentielle sur partition localisée

```

♦ PYTHON — CLIENT CASSANDRA PRODUCTION AVEC ASYNC WRITES

```

# — Python : Client Cassandra avec retry et connection pooling ———
from cassandra.cluster import Cluster, ExecutionProfile, EXEC_PROFILE_DEFAULT
from cassandra.policies import DCAwareRoundRobinPolicy, RetryPolicy,
ExponentialReconnectionPolicy
from cassandra.query import PreparedStatement, ConsistencyLevel
import cassandra.concurrent

# — Configuration production : Consistency Levels —————
profile = ExecutionProfile(
    consistency_level=ConsistencyLevel.LOCAL_QUORUM, # W+R > RF → forte cohérence locale
    # LOCAL_QUORUM = majority des replicas dans le DC local
    # Alternative : ONE (perf max, cohérence éventuelle)
    load_balancing_policy=DCAwareRoundRobinPolicy(local_dc='datacenter-dakar'),

```

```

    retry_policy=RetryPolicy(),
    request_timeout=2.0 # 2s timeout max
)

cluster = Cluster(
    contact_points=['cassandra-node-1', 'cassandra-node-2', 'cassandra-node-3'],
    port=9042,
    execution_profiles={EXEC_PROFILE_DEFAULT: profile},
    reconnection_policy=ExponentialReconnectionPolicy(base_delay=1, max_delay=60),
    protocol_version=5
)
session = cluster.connect('fintech')

# — Prepared Statement : Compilé une fois, exécuté N fois —
INSERT_TXN = session.prepare("""
    INSERT INTO transactions_by_user
    (user_id, transaction_ts, transaction_id, amount, currency, status)
    VALUES (?, ?, uuid(), ?, ?, ?)
    USING TTL 7776000
""")

# — ÉCRITURE BATCH ASYNCHRONE : 10K inserts concurrent —
def insert_batch_async(transactions: list) -> None:
    futures = []
    for txn in transactions:
        future = session.execute_async(
            INSERT_TXN,
            (txn['user_id'], txn['ts'], txn['amount'], txn['currency'], txn['status'])
        )
        futures.append(future)
    # Attendre toutes les futures — non-bloquant jusqu'ici
    errors = []
    for future in futures:
        try:
            future.result()
        except Exception as e:
            errors.append(e)
    if errors:
        raise CassandraWriteException(f'{len(errors)} writes failed')

# Throughput atteint : ~50 000 writes/sec par process Python (16 threads)

```

◆ MongoDB : L'Équilibriste — Documents, Flexibilité et Transactions

MongoDB 7.x en 2026 n'est plus la base NoSQL "schemaless" simple de 2013. C'est un système multi-modèle complet avec transactions ACID multi-documents, time-series collections natives, full-text search intégré, et un pipeline d'agrégation qui rivalise avec SQL en expressivité. Son domaine d'excellence : les données semi-structurées à schéma variable, les catalogues produits, les profils utilisateurs.

* JAVASCRIPT (MQL) — MONGODB AGGREGATION PIPELINE AVANCÉ

```

// — MongoDB : Agrégation Pipeline avancé —
// Cas d'usage : Analyse RFM (Recency, Frequency, Monetary) des clients

db.transactions.aggregate([

    // — Stage 1 : Filtrer les 90 derniers jours —
    { $match: {
        status: 'SUCCESS',
        created_at: { $gte: new Date(Date.now() - 90 * 86400000) }
    }

```

```

    }},

    // — Stage 2 : Grouper par utilisateur —————
    { $group: {
      _id: '$user_id',
      last_transaction_date: { $max: '$created_at' },
      transaction_count:     { $sum: 1 },
      total_spent:           { $sum: '$amount' },
      avg_ticket:            { $avg: '$amount' },
      currencies_used:       { $addToSet: '$currency' }
    }},

    // — Stage 3 : Calculer le score RFM —————
    { $addFields: {
      days_since_last: { $divide: [
        { $subtract: [new Date(), '$last_transaction_date'] },
        86400000 // ms → jours
      ]},
      rfm_recency: { $switch: { branches: [
        { case: { $lte: ['$days_since_last', 7] }, then: 5 },
        { case: { $lte: ['$days_since_last', 30] }, then: 4 },
        { case: { $lte: ['$days_since_last', 60] }, then: 3 },
        { case: { $lte: ['$days_since_last', 90] }, then: 2 },
      ], default: 1 }},
      rfm_frequency: { $switch: { branches: [
        { case: { $gte: ['$transaction_count', 50] }, then: 5 },
        { case: { $gte: ['$transaction_count', 20] }, then: 4 },
        { case: { $gte: ['$transaction_count', 10] }, then: 3 },
        { case: { $gte: ['$transaction_count', 5] }, then: 2 },
      ], default: 1 }},
    }},

    // — Stage 4 : Segmenter les clients —————
    { $addFields: {
      rfm_score: { $add: ['$rfm_recency', '$rfm_frequency'] },
      segment: { $switch: { branches: [
        { case: { $gte: ['$rfm_score', 9] }, then: 'Champions' },
        { case: { $gte: ['$rfm_score', 7] }, then: 'Loyal Customers' },
        { case: { $gte: ['$rfm_score', 5] }, then: 'Potential Loyalists' },
        { case: { $gte: ['$rfm_score', 3] }, then: 'At Risk' },
      ], default: 'Hibernating' }
    }
  }},

    // — Stage 5 : Trier et limiter —————
    { $sort: { total_spent: -1 } },
    { $limit: 10000 },

    // — Stage 6 : Écrire dans une collection de résultats —————
    { $merge: {
      into: 'customer_segments',
      on: '_id',
      whenMatched: 'replace',
      whenNotMatched: 'insert'
    }
  })

  // — Index stratégiques pour performance pipeline —————
  db.transactions.createIndex({ user_id: 1, created_at: -1, status: 1 },
    { name: 'idx_user_date_status', background: true })

```

```
// Index partiel : Seulement les transactions SUCCESS (économise 60% d'espace)
db.transactions.createIndex(
  { user_id: 1, amount: -1 },
  { partialFilterExpression: { status: 'SUCCESS' }, name: 'idx_success_only' }
)
```

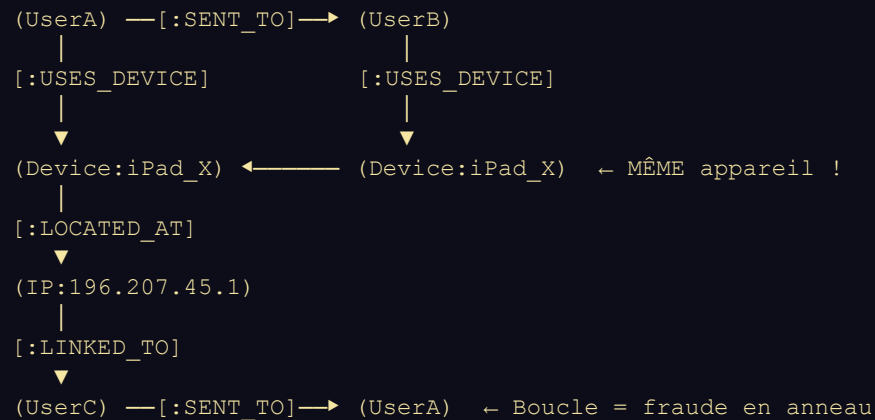
◆ Neo4j : La Puissance des Graphes pour la Détection de Fraude

Neo4j est la base de données de référence pour les problèmes où les RELATIONS entre les entités sont aussi importantes — voire plus — que les entités elles-mêmes. Détection de fraude en anneau, recommandations sociales, analyse de réseau de paiement : Neo4j répond là où SQL et MongoDB deviennent quadratiques en complexité.

NEO4J : MODÈLE GRAPHE — RÉSEAU DE FRAUDE

Nœuds (Nodes)	Relations (Edges)
(:User)	[:SENT_TO]
(:Transaction)	[:USES_DEVICE]
(:Device)	[:LOCATED_AT]
(:IPAddress)	[:OWNS_ACCOUNT]
(:BankAccount)	[:FLAGGED_FOR]

Pattern de Fraude en Anneau (Ring Fraud) :



SQL pour cette query : 5 jointures, ~200ms, $O(n^2)$

Neo4j Cypher : 1 MATCH pattern, ~2ms, $O(\text{depth})$

* CYPHER (NEO4J) — DÉTECTION DE FRAUDE PAR ANALYSE DE GRAPHE

```
// — Cypher : Détection de fraude en anneau —

// — 1. Créer le graphe de transactions —
// MERGE : Crée si n'existe pas, sinon met à jour
MERGE (sender:User {id: $sender_id})
MERGE (receiver:User {id: $receiver_id})
MERGE (device:Device {fingerprint: $device_fp})
MERGE (ip:IPAddress {address: $ip})

CREATE (txn:Transaction {
  id:          $transaction_id,
```

```
    amount:    $amount,
    currency:  $currency,
    timestamp: datetime($timestamp)
  })

CREATE (sender)-[:SENT_TO {amount: $amount}]->(receiver)
CREATE (sender)-[:INITIATED]->(txn)
CREATE (txn)-[:VIA_DEVICE]->(device)
CREATE (txn)-[:FROM_IP]->(ip)

// — 2. Détecter les anneaux de fraude (path length 2-5) —
MATCH path = (start:User)-[:SENT_TO*2..5]->(start)
WHERE ALL(rel IN relationships(path) WHERE rel.amount > 50000)
WITH start, path, length(path) AS ring_length,
     REDUCE(total=0, r IN relationships(path) | total + r.amount) AS total_amount
WHERE total_amount > 500000 // Seuil de signalement
RETURN
    start.id          AS suspicious_user,
    ring_length       AS ring_depth,
    total_amount      AS total_circulated,
    [n IN nodes(path) | n.id] AS ring_participants
ORDER BY total_amount DESC
LIMIT 50

// — 3. Identifier les appareils partagés (collu potential) —
MATCH (u1:User)-[:INITIATED]->(t1:Transaction)-[:VIA_DEVICE]->(d:Device)
    <-[:VIA_DEVICE]->(t2:Transaction)-[:INITIATED]->(u2:User)
WHERE u1.id <> u2.id // Utilisateurs différents
    AND t1.timestamp > datetime() - duration('P7D') // 7 derniers jours
WITH d, collect(DISTINCT u1.id) + collect(DISTINCT u2.id) AS shared_users,
     count(DISTINCT t1) AS shared_transactions
WHERE size(shared_users) >= 3 // 3+ utilisateurs sur même device = suspect
RETURN
    d.fingerprint     AS device_id,
    shared_users       AS users_on_device,
    shared_transactions AS tx_count
ORDER BY size(shared_users) DESC

// — 4. Graph Data Science : PageRank pour influence network —
// Identifier les nœuds les plus influents dans le réseau de fraude
CALL gds.pageRank.stream('fraud-network-graph', {
    maxIterations: 20,
    dampingFactor: 0.85,
    relationshipWeightProperty: 'amount'
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS user_id, score AS influence_score
ORDER BY score DESC
LIMIT 20
// Résultat : Top 20 des comptes les plus centraux dans le réseau de fraude
```

◆ Guide de Décision NoSQL : Cassandra vs MongoDB vs Neo4j

Critère	Cassandra	MongoDB	Neo4j
Modèle de données	Wide-column, time-series	Documents JSON/BSON	Graphe (nœuds + relations)

Scalabilité	Horizontale linéaire ∞	Horizontal sharding	Vertical + Causal Cluster
Volume max testé	Multi-petabyte (Netflix)	Dizaines TB (Sharded)	Milliards de nœuds/reliations
Latence write P99	< 1ms	1-5ms	1-10ms
Transactions ACID	Non (LWT = CAS only)	Oui (multi-document 4.x+)	Oui (ACID natif)
Queries relationnelles	Impossible (pas de join)	Lookup limité (\$lookup)	Natif, ultra-rapide
Schéma	Strict (CQL DDL)	Flexible (schemaless ok)	Semi-flexible
Idéal pour	IoT, logs, time-series	Catalogues, profils, cms	Fraude, reco, réseau social
À éviter pour	Queries analytiques, joins	Write haute fréquence pur	Données tabulaires plates

Synthèse du Chapitre 3 : L'Ingénieur Data God-Mode

La maîtrise du Data Engineering en 2026 ne se résume pas à connaître les APIs de Spark ou les commandes CQL de Cassandra. C'est la capacité à raisonner sur des systèmes qui ingèrent, transforment et servent des pétaoctets de données avec des contraintes simultanées de coût, latence, cohérence et conformité réglementaire. L'ingénieur God-Mode est celui qui peut concevoir l'architecture entière — de la source brute au dashboard analytique — et anticiper les pannes avant qu'elles se produisent.

◉ LES 5 COMMANDEMENTS DU DATA ENGINEER GOD-MODE

- 1. MODÉLISER POUR LES QUERIES : En NoSQL, le schéma découle des questions que vous posez, jamais des entités seules. Inversez votre réflexion.
- 2. LA QUALITÉ AVANT LA VITESSE : Un pipeline rapide qui ingère des données corrompues est pire qu'aucun pipeline. Great Expectations n'est pas optionnel.
- 3. SPARK AQE + BROADCAST = BASE : Activer l'Adaptive Query Execution et utiliser broadcast() sur les petites tables devrait être un réflexe conditionné.
- 4. CAP EST UNE CONTRAINTE PHYSIQUE : Choisir Cassandra pour un système nécessitant des transactions fortes est une faute d'architecture, pas d'implémentation.
- 5. LE PIPELINE EST UN PRODUIT : Versionnez vos schémas (Schema Registry), documentez vos contrats de données, traitez vos DAGs Airflow comme du code de production.

Outil	Force Absolue	Limite à Connaître	Pattern God-Mode
Debezium + Iceberg	CDC sans impact prod	Schéma evolution complexe	Bronze/Silver/Gold layers
Apache Spark AQE	Traitement PB scale	Cold start latence JVM	Broadcast + Pandas UDF
Cassandra	Write < 1ms, scale ∞	Pas de join, modélisation rigide	Partition par query
MongoDB	Flexibilité schéma	Transactions coûteuses	Aggregation Pipeline
Neo4j	Relations en O(1)	Scale horizontal limité	Cypher + GDS PageRank

Chapitre 4 → Machine Learning en Production : Feature Stores, MLOps & LLM Engineering